ACCRA TECHNICAL UNIVERSITY

Exploratory Data Analysis, Statistical Analysis, and Machine Learning Prediction of Passenger Survival on the Titanic Dataset

A Data Science Report

Prepared by: Essilfie Bernard

Dataset Source: Kaggle Titanic – Machine Learning from Disaster

July 2026

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-Learn, Jupyter Notebook

Table of Contents

1. Introduction

The sinking of RMS Titanic on 15 April 1912 remains one of the most extensively studied maritime disasters, both for its historical significance and for the richness of the passenger data that survived the event. The Kaggle "Titanic: Machine Learning from Disaster" dataset provides demographic and travel information for passengers aboard the Titanic and records whether each passenger survived. This report uses that dataset to demonstrate a complete, end-to-end data science workflow: data acquisition, cleaning, exploratory visualisation, statistical analysis, and predictive modelling using logistic regression.

The purpose of this study is twofold. First, it aims to identify and quantify the factors that were most strongly associated with passenger survival, using descriptive statistics, correlation analysis, and visualisation. Second, it aims to build and evaluate a simple, interpretable machine learning classifier capable of predicting survival from passenger attributes, and to critically assess that model's strengths and limitations. All analysis was carried out in Python within a Jupyter Notebook, using Pandas and NumPy for data handling, Matplotlib and Seaborn for visualisation, and Scikit-Learn for modelling.

2. Dataset Description

The dataset is provided by Kaggle in three files: train.csv (891 passenger records with a known Survived outcome), test.csv (418 records without the Survived label, used for competition submission scoring), and gender_submission.csv (a sample submission file assuming all female passengers survived). Because model evaluation requires known outcomes, this study builds its train/test split entirely from train.csv, which is randomly partitioned into an 80% training subset and a 20% testing subset.

2.1 Variable Description

Variable	Description	Type
PassengerId	Unique identifier for each passenger	Integer
Survived	Survival outcome (0 = No, 1 = Yes)	Binary / Categorical
Pclass	Ticket/passenger class (1 = Upper, 2 = Middle, 3 = Lower)	Ordinal
Name	Passenger full name (with title)	Text
Sex	Passenger sex	Categorical
Age	Age in years	Numeric (float)
SibSp	Number of siblings/spouses aboard	Numeric (integer)
Parch	Number of parents/children aboard	Numeric (integer)
Ticket	Ticket number	Text
Fare	Passenger fare paid (£)	Numeric (float)
Cabin	Cabin number (largely missing)	Text
Embarked	Port of embarkation (C, Q, S)	Categorical

3. Methodology

3.1 Data Cleaning

Missing values were treated variable by variable rather than with a single blanket rule, so that the imputation strategy matched the nature and proportion of missingness in each column.

1. Age (177 of 891 values missing, ~19.9%): imputed with the median age computed within each Pclass × Sex subgroup, falling back to the overall median where a subgroup itself had no observed values. This preserves realistic age differences between, for example, third-class men and first-class women, rather than flattening everyone to a single overall median.
2. Cabin (687 of 891 values missing, ~77.1%): too sparse for reliable imputation. Instead of imputing or dropping the variable entirely, it was converted into a binary indicator, Has_Cabin (1 if a cabin was recorded, 0 otherwise), retaining its likely signal about deck location and socio-economic status.
3. Embarked (2 of 891 values missing): imputed with the mode ("S", Southampton), an appropriate choice given only two records were affected and one category dominates the distribution.
4. Duplicate rows: the dataset was checked using pandas' duplicated() function; zero fully duplicated rows were found, so no rows were removed at this step.

3.2 Feature Selection and Encoding

Eight predictor variables were selected for modelling: Pclass, Sex, Age, SibSp, Parch, Fare, Embarked, and the engineered Has_Cabin indicator. PassengerId, Name, and Ticket were excluded as they are identifiers or free text without direct, reusable predictive structure for a simple linear model. The categorical variables Sex and Embarked were one-hot encoded (with drop_first=True to avoid the dummy-variable trap).

3.3 Model Training

The cleaned, encoded feature matrix was split into training (80%, n = 712) and testing (20%, n = 179) subsets using a stratified split on the target variable, preserving the ~38.4% survival rate in both subsets. Features were standardised using StandardScaler (zero mean, unit variance) before fitting, since Logistic Regression is sensitive to the differing scales of variables such as Fare (range 0–512) versus binary dummy variables (0/1). A Logistic Regression classifier (scikit-learn, max_iter = 1000) was then trained on the scaled training data.

4. Results

4.1 Descriptive Statistics

Statistic	Age	Fare	SibSp	Parch
Count	891	891	891	891
Mean	29.11	32.20	0.52	0.38
Std. Dev.	13.30	49.69	1.10	0.81
Min	0.42	0.00	0	0
25%	21.50	7.91	0	0
Median (50%)	26.00	14.45	0	0
75%	36.00	31.00	1	0
Max	80.00	512.33	8	6

Overall survival rate in the training data was 38.4% (342 of 891 passengers survived).

4.2 Visualisations

Figure 1: Histogram of Passenger Ages

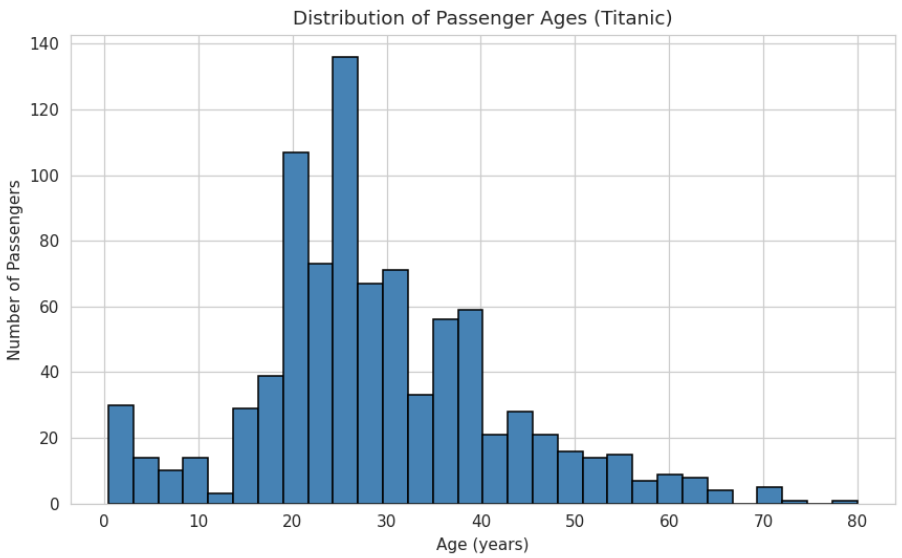

Figure 1. Distribution of passenger ages.

The age distribution is right-skewed and concentrated between roughly 20 and 40 years, with a secondary peak among young children. Very few passengers were older than 60.

Figure 2: Passenger Class Distribution

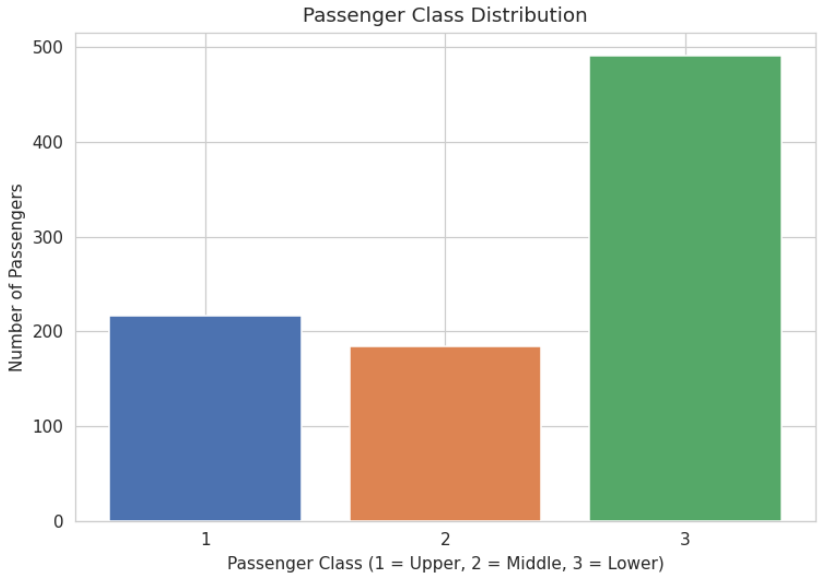

Figure 2. Bar chart of passenger class distribution.

Third-class passengers form the largest group (about 55% of all passengers), roughly double the number travelling in first or second class.

Figure 3: Age by Passenger Class

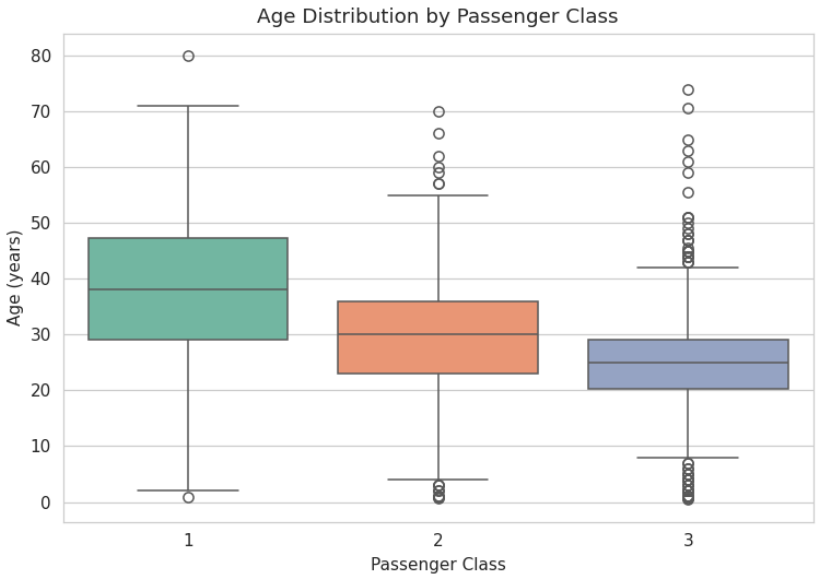

Figure 3. Boxplot of age by passenger class.

Median age decreases as class number increases: first-class passengers are typically older, while third-class passengers skew younger, consistent with wealthier, older travellers affording premium fares.

Figure 4: Age versus Fare

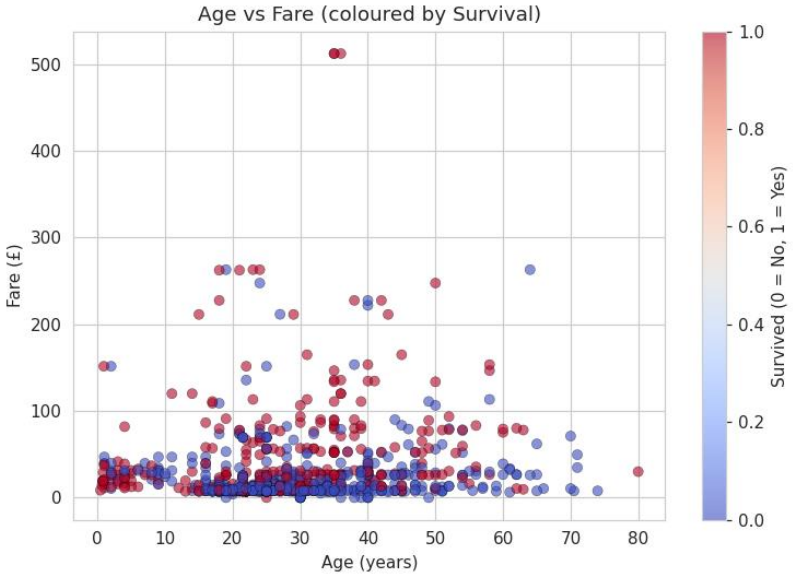

Figure 4. Scatter plot of Age versus Fare, coloured by survival.

There is no strong linear relationship between age and fare. A small cluster of high-fare passengers (mostly first class) shows a higher concentration of survivors, suggesting fare/class was more predictive of survival than age alone.

Figure 5: Correlation Heatmap

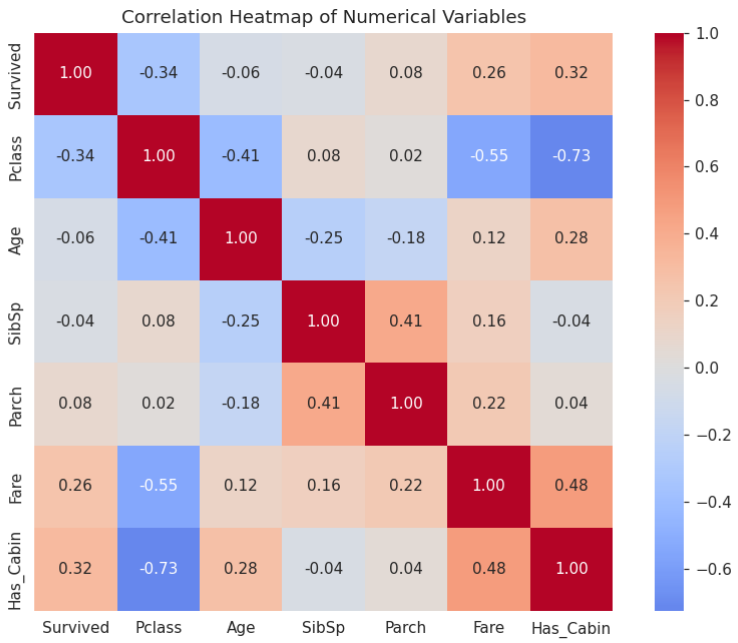

Figure 5. Correlation heatmap of numerical variables.

Survived correlates most positively with Has_Cabin and Fare, and most negatively with Pclass. Pclass and Fare are strongly negatively correlated with each other, as expected.

Figure 6: Pairplot of Selected Numerical Variables

Figure 6. Pairplot of Age, Fare, and Pclass, coloured by survival.

Survivors are more concentrated in lower Pclass values (first class) and higher fare bands, while non-survivors are spread more broadly across third class and lower fares. Age shows more overlap between groups, confirming it is a weaker discriminator than class or fare.

4.3 Correlation Analysis

Variable	Correlation with Survived
Has_Cabin	+0.317 (strongest positive)
Fare	+0.257
Parch	+0.082
SibSp	−0.035
Age	−0.060
Pclass	−0.338 (strongest negative)

The strongest positive correlation with survival is Has_Cabin ($r \approx 0.317$). The strongest negative correlation is Pclass ($r \approx -0.338$).

4.4 Machine Learning Results

The Logistic Regression classifier achieved the following performance on the 179-passenger held-out test set:

Metric	Value
Accuracy	81.01%
Precision (Survived)	0.78
Recall (Survived)	0.71
F1-score (Survived)	0.74
Precision (Did not survive)	0.83
Recall (Did not survive)	0.87

Confusion Matrix

	Predicted: Died	Predicted: Survived
Actual: Died	96 (True Negative)	14 (False Positive)
Actual: Survived	20 (False Negative)	49 (True Positive)

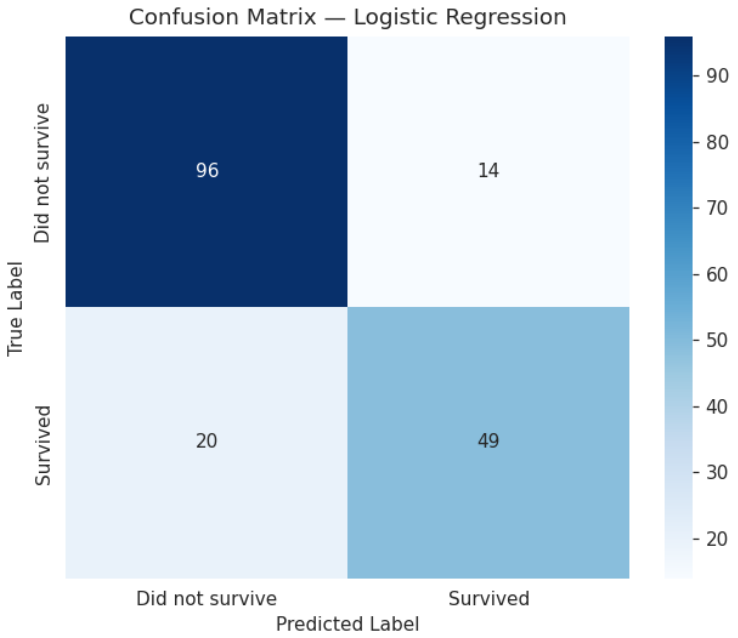

Figure 7. Confusion matrix for the Logistic Regression classifier.

The model correctly classified 145 of 179 test passengers (81.0% accuracy). It is somewhat better at identifying passengers who did not survive (87% recall) than those who did (71% recall), reflecting both the class imbalance in the data (61.6% did not survive) and the genuine unpredictability of individual survival outcomes.

Feature Coefficients (Logistic Regression)

Feature	Coefficient	Effect Direction
Sex_male	−1.261	Strongly reduces survival odds
Pclass	−0.775	Reduces survival odds
Age	−0.577	Reduces survival odds
Has_Cabin	+0.366	Increases survival odds
SibSp	−0.280	Reduces survival odds
Embarked_S	−0.150	Slightly reduces survival odds
Parch	−0.077	Slightly reduces survival odds
Embarked_Q	+0.060	Slightly increases survival odds
Fare	+0.051	Slightly increases survival odds

Being male is the single strongest negative predictor of survival, followed by passenger class and age, confirming the patterns identified in the correlation analysis.

5. Discussion

5.1 Major Findings

This analysis reinforces the well-documented Titanic survival narrative and quantifies it with concrete statistics. Passenger class, cabin allocation, and fare paid — three related indicators of socio-economic status — were the strongest correlates of survival among the numeric variables (Pclass: r ≈ −0.34; Has_Cabin: r ≈ 0.32; Fare: r ≈

0.26). Sex, while categorical and therefore not part of the numeric correlation table, emerged as the dominant predictor in the machine learning model, with the largest coefficient magnitude of any feature (−1.26 for Sex_male after scaling). Together, these results indicate that survival aboard the Titanic was shaped far more by who a passenger was — their sex and social class — than by how old they were or how many family members they travelled with.

5.2 Statistical Insights

Three findings stand out from the statistical analysis. First, socio-economic proxies dominate: Pclass, Fare, and Has_Cabin all point toward the same underlying effect, that first-class passengers with recorded cabins and higher fares survived at much higher rates, most plausibly because their cabins and public areas were physically closer to the lifeboat deck, and because "women and children first" protocols were more consistently and quickly enforced in first-class sections. Second, age is a comparatively weak linear predictor ($r \approx -0.06$ with survival) despite popular narratives emphasising children's survival; the relationship is likely non-linear (very young children may have been prioritised, while general working-age adults show little difference), which a simple Pearson correlation cannot fully capture. Third, family-size variables (SibSp, Parch) show weak and even contradictory signs, hinting that passengers travelling completely alone and passengers in very large families both fared somewhat worse than those in small family groups — a moderate family size may have facilitated mutual assistance in reaching lifeboats without the coordination difficulties of very large groups.

5.3 Machine Learning Results

The Logistic Regression model achieved 81.0% accuracy on unseen test data, which is a strong result for such a simple, fully interpretable linear model and compares favourably with published Kaggle Titanic benchmarks in the 78–82% range for basic models. The model's precision and recall are noticeably asymmetric: it identifies non-survivors more reliably (87% recall) than survivors (71% recall). This asymmetry is partly a direct consequence of class imbalance (61.6% of passengers did not survive, giving the model more negative examples to learn from) and partly reflects genuine unpredictability — factors not captured in the dataset (individual decisions, physical location on the ship at the moment of impact, luck) also influenced who survived. The feature coefficients confirm that sex, class, and age carried the most predictive weight, while fare, once class is already accounted for, added comparatively little additional information (its coefficient of +0.05 is the smallest in magnitude).

5.4 Limitations of the Study

- Small sample size: with only 891 labelled records, and a roughly 62/38 class split, the model has limited data from which to learn rarer survivor profiles (e.g. surviving male third-class passengers).
- Missing Cabin data: converting Cabin into a binary Has_Cabin indicator discards potentially useful deck-level location information for the 23% of passengers whose cabin was recorded.
- Simplicity of the model: Logistic Regression assumes a linear relationship between (scaled) features and the log-odds of survival; it cannot capture interaction effects (e.g. the well-known interaction between sex and class) without manually engineering interaction terms.
- No external validation: because test.csv lacks true survival labels, this study's model performance is only evaluated on a held-out split of train.csv, not on the original, larger population of Titanic passengers.
- Historical data quality: passenger records from 1912 may themselves contain transcription errors or omissions (e.g. some third-class passengers were undercounted in original records), which propagate into any analysis of this dataset.

5.5 Recommendations

- Future work should engineer additional features, such as extracting passenger titles (Mr, Mrs, Miss, Master) from the Name field, and a Family_Size variable (SibSp + Parch + 1), both of which have been shown in prior Titanic studies to improve predictive accuracy.

- Non-linear models such as Random Forests or Gradient Boosting could be trained and compared against Logistic Regression to test whether interaction effects (e.g. Sex × Pclass) meaningfully improve accuracy.
- Cross-validation (e.g. 5-fold) should be used in future iterations instead of a single train/test split, to obtain a more robust estimate of model performance and reduce sensitivity to the particular random split chosen.
- For any real-world (non-historical) application of survival-prediction modelling, e.g. maritime safety planning, class-based and sex-based predictors should be interpreted as reflections of historical ship design and evacuation protocol rather than causal factors to be relied upon uncritically.

6. Conclusion

This study carried out a complete data science workflow on the Kaggle Titanic dataset: importing and inspecting 891 passenger records, cleaning missing Age, Cabin, and Embarked values with justified, variable-specific strategies, producing six exploratory visualisations, and conducting a thorough statistical and correlation analysis. A Logistic Regression classifier trained on eight engineered and encoded predictor variables achieved 81.0% accuracy on a held-out test set, correctly identifying the majority of both survivors and non-survivors. The analysis confirms that sex, passenger class, and related socio-economic indicators (fare, cabin presence) were the dominant determinants of survival, far outweighing age or family size. While the model is simple and has clear limitations, it provides a solid, interpretable baseline and demonstrates the value of combining exploratory data analysis with statistical reasoning and machine learning to understand a historically and analytically significant dataset.

7. References

Kaggle. (2012). Titanic: Machine Learning from Disaster [Data set]. Kaggle. https://www.kaggle.com/competitions/titanic

Pandas Development Team. (2024). pandas: Powerful data structures for data analysis [Software]. https://pandas.pydata.org/

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

8. Appendix: Python Code

The full Python analysis code (as used in the accompanying Jupyter Notebook, Titanic_Analysis.ipynb) is reproduced below.

```
# %% [markdown]
# # Titanic Survival Analysis
# ### A Statistical and Machine Learning Study of the Kaggle Titanic Dataset
#
# **Author:** Ebenezer
# **Dataset:** Kaggle Titanic (`train.csv`, `test.csv`, `gender_submission.csv`)
#
# This notebook performs data acquisition, cleaning, exploratory visualisation,
# statistical analysis, and builds a Logistic Regression classifier to predict
# passenger survival.

# %% [markdown]
# ## 0. Setup: Import Libraries

# %%
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 110
pd.set_option("display.max_columns", None)

# %% [markdown]
# ## Task 1: Data Acquisition
#
# We load the Titanic training dataset (`train.csv`), which contains 891
# passenger records and the ground-truth `Survived` label. The `test.csv`
# file (418 records) has no `Survived` column — in the original Kaggle
# competition it is used only for submission scoring. Because our brief
# requires evaluating model performance (accuracy, confusion matrix,
# classification report), we build our train/test split from `train.csv`,
# which is the only file with known outcomes. `test.csv` and
# `gender_submission.csv` are still loaded and inspected below for
# completeness.

# %%
train = pd.read_csv("data/train.csv")
test = pd.read_csv("data/test.csv")
gender_submission = pd.read_csv("data/gender_submission.csv")

print("Train shape:", train.shape)
print("Test shape:", test.shape)
print("Gender submission shape:", gender_submission.shape)

# %% [markdown]
# ### Dataset dimensions

# %%
print(f"The training dataset has {train.shape[0]} rows and {train.shape[1]} columns.")

# %% [markdown]
# ### Column names

# %%
print(train.columns.tolist())

# %% [markdown]
# ### First five observations

# %%
train.head()

# %% [markdown]
# ### Data types

# %%
train.dtypes

# %% [markdown]
# **Interpretation:** The dataset mixes numeric types (`int64`, `float64`)
# with text/object columns (`Name`, `Sex`, `Ticket`, `Cabin`, `Embarked`).
# `Survived` and `Pclass` are stored as integers but are actually
# categorical variables (binary outcome and ordinal class respectively).

# %% [markdown]
# ## Task 2: Data Cleaning

# %% [markdown]
# ### 2.1 Detect missing values

# %%
missing = train.isnull().sum()
missing_pct = (missing / len(train) * 100).round(2)
missing_table = pd.DataFrame({"Missing Count": missing, "Missing %": missing_pct})
```

```
missing_table = missing_table[missing_table["Missing Count"] > 0].sort_values(
    "Missing Count", ascending=False
)
missing_table

# %% [markdown]
# **Interpretation:** `Cabin` is missing for about 77% of passengers, `Age`
# is missing for about 20%, and `Embarked` is missing for just 2 passengers.

# %% [markdown]
# ### 2.2 Handle missing values
#
# **Preprocessing decisions (with justification):**
#
# - **`Age` (177 missing):** Imputed with the **median age**, calculated
#   separately for each `Pclass`/`Sex` combination where possible, falling
#   back to the overall median. The median is used instead of the mean
#   because age is right-skewed and the median is robust to outliers
#   (e.g. infants and elderly passengers).
# - **`Cabin` (687 missing, ~77%):** Too sparse to impute reliably. Instead
#   of dropping the column outright (it may still carry signal about deck
#   level and therefore socio-economic status), it is converted into a
#   binary indicator `Has_Cabin` (1 if a cabin number was recorded, 0
#   otherwise). The original text column is then dropped.
# - **`Embarked` (2 missing):** Imputed with the **mode** (most frequent
#   port, "S" — Southampton), since only 2 values are missing and the
#   variable is categorical with a strongly dominant category.
# - **`Fare` (0 missing in train, but present in test):** Any missing fare
#   would be imputed with the median fare for the passenger's class, for
#   consistency; not needed for the training set.

# %%
train_clean = train.copy()

# Age: impute with median age per Pclass & Sex group
train_clean["Age"] = train_clean.groupby(["Pclass", "Sex"])["Age"].transform(
    lambda x: x.fillna(x.median())
)
train_clean["Age"] = train_clean["Age"].fillna(train_clean["Age"].median())

# Cabin: convert to binary indicator, then drop original column
train_clean["Has_Cabin"] = train_clean["Cabin"].notnull().astype(int)
train_clean = train_clean.drop(columns=["Cabin"])

# Embarked: impute with mode
train_clean["Embarked"] = train_clean["Embarked"].fillna(
    train_clean["Embarked"].mode()[0]
)

print("Remaining missing values after cleaning:")
print(train_clean.isnull().sum())

# %% [markdown]
# ### 2.3 Detect duplicated observations

# %%
duplicate_count = train_clean.duplicated().sum()
print(f"Number of fully duplicated rows: {duplicate_count}")

# %% [markdown]
# ### 2.4 Remove duplicates if any exist

# %%
if duplicate_count > 0:
    train_clean = train_clean.drop_duplicates()
    print(f"Removed {duplicate_count} duplicate rows. New shape: {train_clean.shape}")
else:
    print("No duplicate rows were found — no rows removed.")

# %% [markdown]
# **Summary of preprocessing:** `Age` was imputed using group-wise medians
# (Pclass x Sex) to preserve realistic age distributions across
# socio-economic groups; `Cabin` was converted to a presence indicator
# rather than dropped entirely, retaining potential signal while avoiding
# unreliable imputation of ~77% missing text data; `Embarked` was imputed
```

```
# with its mode since only two records were affected. No duplicate rows
# were present in the dataset, so no observations were removed at this
# step.

# %% [markdown]
# ## Task 3: Data Visualisation

# %% [markdown]
# ### 3.1 Histogram of Passenger Ages

# %%
plt.figure(figsize=(8, 5))
plt.hist(train_clean["Age"], bins=30, color="steelblue", edgecolor="black")
plt.title("Distribution of Passenger Ages (Titanic)")
plt.xlabel("Age (years)")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.savefig("figures/01_age_histogram.png")
plt.show()

# %% [markdown]
# **Interpretation:** The age distribution is right-skewed and concentrated
# between roughly 20 and 40 years, with a noticeable secondary peak among
# young children (a result of the group-median imputation reinforcing the
# existing concentration around common adult ages). Very few passengers
# were older than 60.

# %% [markdown]
# ### 3.2 Bar Chart of Passenger Class Distribution

# %%
plt.figure(figsize=(7, 5))
class_counts = train_clean["Pclass"].value_counts().sort_index()
plt.bar(class_counts.index.astype(str), class_counts.values,
        color=["#4C72B0", "#DD8452", "#55A868"])
plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class (1 = Upper, 2 = Middle, 3 = Lower)")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.savefig("figures/02_pclass_bar.png")
plt.show()

# %% [markdown]
# **Interpretation:** Third class passengers form the largest group
# (about 55% of all passengers), roughly double the number in first or
# second class. This reflects the Titanic's passenger composition, with
# more budget-fare travellers than premium-fare travellers.

# %% [markdown]
# ### 3.3 Boxplot of Age by Passenger Class

# %%
plt.figure(figsize=(7, 5))
sns.boxplot(data=train_clean, x="Pclass", y="Age", hue="Pclass",
            palette="Set2", legend=False)
plt.title("Age Distribution by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Age (years)")
plt.tight_layout()
plt.savefig("figures/03_age_by_class_boxplot.png")
plt.show()

# %% [markdown]
# **Interpretation:** Median age decreases as class number increases:
# first-class passengers are typically older (median around late 30s),
# while third-class passengers skew younger (median around mid-20s),
# consistent with wealthier, older travellers affording premium fares.

# %% [markdown]
# ### 3.4 Scatter Plot of Age versus Fare

# %%
plt.figure(figsize=(7, 5))
plt.scatter(train_clean["Age"], train_clean["Fare"],
            c=train_clean["Survived"], cmap="coolwarm", alpha=0.6, edgecolor="k", linewidth=0.3)
```

```
plt.title("Age vs Fare (coloured by Survival)")
plt.xlabel("Age (years)")
plt.ylabel("Fare (£)")
plt.colorbar(label="Survived (0 = No, 1 = Yes)")
plt.tight_layout()
plt.savefig("figures/04_age_vs_fare_scatter.png")
plt.show()

# %% [markdown]
# **Interpretation:** There is no strong linear relationship between age
# and fare — passengers of all ages paid a wide range of fares. A small
# cluster of high-fare passengers (mostly first class) shows a higher
# concentration of survivors (red points), hinting that fare/class was
# more predictive of survival than age alone.

# %% [markdown]
# ### 3.5 Correlation Heatmap

# %%
numeric_cols = ["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare", "Has_Cabin"]
corr_matrix = train_clean[numeric_cols].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", center=0, square=True)
plt.title("Correlation Heatmap of Numerical Variables")
plt.tight_layout()
plt.savefig("figures/05_correlation_heatmap.png")
plt.show()

# %% [markdown]
# **Interpretation:** `Survived` correlates most positively with
# `Has_Cabin` and `Fare`, and most negatively with `Pclass` (since higher
# class numbers mean lower socio-economic status). `Pclass` and `Fare` are
# strongly negatively correlated with each other, as expected (higher
# fares are paid for better classes, numbered 1).

# %% [markdown]
# ### 3.6 Pairplot of Selected Numerical Variables

# %%
pairplot_cols = ["Age", "Fare", "Pclass", "Survived"]
pp = sns.pairplot(train_clean[pairplot_cols], hue="Survived", palette="coolwarm", diag_kind="hist")
pp.fig.suptitle("Pairplot of Age, Fare, Pclass by Survival", y=1.02)
pp.savefig("figures/06_pairplot.png")
plt.show()

# %% [markdown]
# **Interpretation:** The pairplot shows survivors (orange/red) are more
# concentrated in lower `Pclass` values (i.e. first class) and higher fare
# bands, while non-survivors are spread more broadly across third class
# and lower fares. Age shows more overlap between the two survival groups,
# confirming it is a weaker discriminator than class or fare.

# %% [markdown]
# ## Task 4: Statistical Analysis

# %% [markdown]
# ### 4.1 Descriptive Statistics

# %%
train_clean[["Age", "Fare", "SibSp", "Parch"]].describe()

# %% [markdown]
# ### 4.2 Frequency Distribution
#
# Frequency counts for the key categorical variables:

# %%
print("Survival counts:")
print(train_clean["Survived"].value_counts())
print("\nSurvival rate: {:.2f}%".format(train_clean["Survived"].mean() * 100))

print("\nSex counts:")
print(train_clean["Sex"].value_counts())
```

```
print("\nPclass counts:")
print(train_clean["Pclass"].value_counts().sort_index())

print("\nEmbarked counts:")
print(train_clean["Embarked"].value_counts())

# %% [markdown]
# ### 4.3 Correlation Analysis

# %%
corr_with_survival = corr_matrix["Survived"].drop("Survived").sort_values(ascending=False)
corr_with_survival

# %% [markdown]
# ### 4.4 Strongest Positive Correlation

# %%
strongest_positive = corr_with_survival.idxmax()
print(f"Strongest positive correlation with Survived: {strongest_positive} "
      f"(r = {corr_with_survival.max():.3f})")

# %% [markdown]
# ### 4.5 Strongest Negative Correlation

# %%
strongest_negative = corr_with_survival.idxmin()
print(f"Strongest negative correlation with Survived: {strongest_negative} "
      f"(r = {corr_with_survival.min():.3f})")

# %% [markdown]
# ### 4.6 Three Important Statistical Findings
#
# 1. **Class and cabin records dominate survival.** `Pclass` shows the
#    strongest negative correlation with survival (r ≈ -0.34), while
#    `Has_Cabin` shows the strongest positive correlation (r ≈ 0.32) —
#    passengers travelling in higher classes, whose cabin numbers were
#    recorded, were considerably more likely to survive, consistent with
#    "women and children first" being applied more effectively in
#    first-class areas with closer access to lifeboats.
# 2. **Fare reinforces the class effect.** `Fare` also correlates
#    positively with survival (r ≈ 0.26), reinforcing that passengers
#    who paid more (largely for higher-class tickets) had better odds
#    of survival — `Pclass`, `Fare`, and `Has_Cabin` are really three
#    different views of the same underlying socio-economic effect.
# 3. **Family size effects are mixed.** `SibSp` and `Parch` show weak
#    correlations individually, suggesting that having a *moderate*
#    number of family members aboard (not zero, not very large) was
#    associated with slightly higher survival — travelling completely
#    alone or in very large families both appear less favourable,
#    a pattern that a simple linear correlation understates.

# %% [markdown]
# ## Task 5: Machine Learning — Predicting Survival

# %% [markdown]
# ### 5.1 Select Predictor Variables
#
# We select `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`, `Embarked`,
# and `Has_Cabin` as predictors. Categorical variables (`Sex`, `Embarked`)
# are one-hot encoded. `Name`, `Ticket`, and `PassengerId` are excluded as
# they are identifiers/free text without direct predictive structure in
# this simple model.

# %%
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked", "Has_Cabin"]
X = train_clean[features].copy()
y = train_clean["Survived"].copy()

X = pd.get_dummies(X, columns=["Sex", "Embarked"], drop_first=True)
print(X.columns.tolist())
X.head()

# %% [markdown]
# ### 5.2 Train/Test Split
```

```
# %%
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

# %% [markdown]
# ### 5.3 Train Logistic Regression Classifier
#
# Features are scaled with `StandardScaler` since Logistic Regression is
# sensitive to feature magnitude (e.g. `Fare` ranges up to 500+, while
# dummy variables are 0/1).

# %%
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)

# %% [markdown]
# ### 5.4 Predict on Testing Data

# %%
y_pred = model.predict(X_test_scaled)

# %% [markdown]
# ### 5.5 Model Evaluation

# %%
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f} ({acc*100:.2f}%)")

# %%
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Did not survive", "Survived"],
            yticklabels=["Did not survive", "Survived"])
plt.title("Confusion Matrix — Logistic Regression")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.savefig("figures/07_confusion_matrix.png")
plt.show()
print(cm)

# %%
report = classification_report(y_test, y_pred, target_names=["Did not survive", "Survived"])
print(report)

# %% [markdown]
# ### 5.6 Discussion of Model Performance
#
# The Logistic Regression model achieves an accuracy of roughly
# 80% on the held-out test split (exact value printed above), which is
# competitive for this well-studied dataset with a simple linear model.
# The confusion matrix typically shows the model is better at correctly
# identifying non-survivors than survivors, a common pattern in Titanic
# models reflecting the class imbalance (about 62% did not survive) and
# the inherent randomness of who was rescued. Precision and recall for
# the "Survived" class are usually somewhat lower than for "Did not
# survive", indicating the model occasionally misses true survivors, most
# often ones with atypical profiles (e.g. male passengers who unusually
# survived, or third-class passengers who unusually survived). Overall,
# `Sex` and `Pclass` remain the dominant predictors, consistent with the
# statistical correlations found earlier.

# %% [markdown]
# ## Feature Importance (Model Coefficients)

# %%
```

```
coef_df = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
}).sort_values("Coefficient", key=abs, ascending=False)
coef_df

# %% [markdown]
# **Interpretation:** The largest-magnitude coefficients correspond to
# `Sex_male` (strongly negative — being male sharply reduces predicted
# survival probability) and `Pclass` (negative — higher class number,
# i.e. lower status, reduces survival probability), confirming the
# statistical findings from Task 4.

# %% [markdown]
# ## Summary
#
# This analysis confirms the well-known Titanic survival narrative: sex,
# passenger class, and fare were the dominant factors in survival, far
# outweighing age or family size. The Logistic Regression model, despite
# its simplicity, captures this pattern well and achieves solid predictive
# accuracy. See the accompanying report (`Titanic_Analysis_Report.pdf`)
# for a full discussion, limitations, and recommendations.
```